UNIVERSITY OF
Southampton
School of Electronics
and Computer Science

Comp1004: Loops and Arrays II

Collections and Iterators

Coming Up

- Arrays vs. ArrayLists
  - Declaration
  - Insertion
  - Access
  - Removal
- A Brief Introduction to Generics
  - Autoboxing and unboxing
- Iterator objects

Arrays vs. ArrayLists

Problems with Arrays

- They don’t change size
- It’s a pain adding new elements if you don’t know how many are there already
- You have to use indexes
- ArrayIndexOutOfBoundsException

ArrayList to the rescue!

- Arrays are built into the Java language (a bit like primitives)
- But Java also have a **library** of helpful classes you can use for free
- These are not part of the language, but are included with every JVM
- ArrayList is one of these library classes

Array vs ArrayList

Array	ArrayList

Array vs ArrayList

Array	ArrayList
They don’t change size	Changes size as you add elements

Array vs ArrayList

Array	ArrayList
They don’t change size	Changes size as you add elements
It’s a pain adding new elements if you don’t know how many are there already	ArrayList has an add() method and takes care of its size itself

Array vs ArrayList

Array	ArrayList
They don’t change size	Changes size as you add elements
It’s a pain adding new elements if you don’t know how many are there already	ArrayList has an add() method and takes care of its size itself
You have to use indexes	You can use indexes if you want, but you don’t have to

Array vs ArrayList

Array	ArrayList
They don’t change size	Changes size as you add elements
It’s a pain adding new elements if you don’t know how many are there already	ArrayList has an add() method and takes care of its size itself
You have to use indexes	You can use indexes if you want, but you don’t have to
ArrayIndexOutOfBoundsException	Still thrown by ArrayList. Hey, it’s a fact of life, okay?

Array vs ArrayList

Array	ArrayList

Declaration Insertion Access Removal

```
Cat[] catArray;
catArray = new Cat[10];

catArray[0] = moggy1;
catArray[1] = moggy2;

callMethodOn(catArray[1]);

catArray[0] = null;
```

```
ArrayList  catAList;
catAList = new ArrayList();

catAList.add(moggy1);
catAList.add(moggy2);

callMethodOn(catAList.get(1));

catAList.remove(moggy1);
```

ArrayList Advantages

- Arrays are useful for simple small tasks
- ArrayLists are better for more complex tasks
  - They grow and shrink when you add and remove things – arrays are fixed size
  - They have many useful methods...
- Check out the API:
  - Application Programming Interface
  - http://download.oracle.com/javase/6/docs/api/
  - type ‘java api’ into google

Here’s a **bit** of the method summary for ArrayList...

Brief Introduction to Generics

Spot the Problem…

```
ArrayList kennel = new ArrayList();

kennel.add(new Dog(“Rover”));
kennel.add(new Dog(“Fido”));
kennel.add(new Dog(“Patch”));
kennel.add(new Cat(“Mr Tiddles”));


for(int i = 0; i < kennel.size(); i++) {
   kennel.get(i).bark();
}
```

Spot the Problem…

```
ArrayList kennel = new ArrayList();

kennel.add(new Dog("Rover"));
kennel.add(new Dog("Fido"));
kennel.add(new Dog("Patch"));
kennel.add(new Cat("Mr Tiddles"));


for(int i = 0; i < kennel.size(); i++) {
    kennel.get(i).bark();
}
```

ArrayLists store objects of any type

Which means we can mix up the types of objects in the ArrayList

Which may cause problems later if we make assumptions about what is in there!

Spot the Problem…

```
ArrayList kennel = new ArrayList();

kennel.add(new Dog("Rover"));
kennel.add(new Dog("Fido"));
kennel.add(new Dog("Patch"));
kennel.add(new Cat("Mr Tiddles"));


for(int i = 0; i < kennel.size(); i++) {
    kennel.get(i).bark();
}
```

ArrayLists store objects of any type

Which means we can mix up the types of objects in the ArrayList

Which may cause problems later if we make assumptions about what is in there!

In fact this code will not compile, because Java does not know what is in the ArrayList, and therefore will not let you call **bark** on it

Solving the Problem…

```
ArrayList kennel = new ArrayList();

kennel.add(new Dog(“Rover”));
kennel.add(new Dog(“Fido”));
kennel.add(new Dog(“Patch”));
kennel.add(new Cat(“Mr Tiddles”));


for(int i = 0; i < kennel.size(); i++) {
   kennel.get(i).bark();
}
```

Solving the Problem…

```
ArrayList kennel = new ArrayList();

kennel.add(new Dog("Rover"));
kennel.add(new Dog("Fido"));
kennel.add(new Dog("Patch"));
kennel.add(new Cat("Mr Tiddles"));


for(int i = 0; i < kennel.size(); i++) {
   if(kennel.get(i) instanceof Dog) {
        Dog d = (Dog)kennel.get(i);
        d.bark();
   }
}
```

One option is to test what is in there using **instanceof**, and if it's a Dog we can tell the compiler.

This is called **typecasting**

Solving the Problem…

```
ArrayList kennel = new ArrayList();

kennel.add(new Dog("Rover"));
kennel.add(new Dog("Fido"));
kennel.add(new Dog("Patch"));
kennel.add(new Cat("Mr Tiddles"));


for(int i = 0; i < kennel.size(); i++) {
   if(kennel.get(i) instanceof Dog) {
        Dog d = (Dog)kennel.get(i);
        d.bark();
   }
}
```

One option is to test what is in there using **instanceof**, and if it's a Dog we can tell the compiler.

This is called **typecasting**

Makes my inner software engineer cringe!

instanceof is a tool of last resort. If you've had to use it it probably means you're program is not designed particularly well.

Solving the Problem…

```
ArrayList kennel = new ArrayList();

kennel.add(new Dog(“Rover”));
kennel.add(new Dog(“Fido”));
kennel.add(new Dog(“Patch”));
kennel.add(new Cat(“Mr Tiddles”));


for(int i = 0; i < kennel.size(); i++) {
   kennel.get(i).bark();
}
```

It would be better if we could ensure that the ArrayList **only** contained Dogs in the first place

Solving the Problem…

```
ArrayList<Dog> kennel = new ArrayList<Dog>();

kennel.add(new Dog("Rover"));
kennel.add(new Dog("Fido"));
kennel.add(new Dog("Patch"));
kennel.add(new Cat("Mr Tiddles"));


for(int i = 0; i < kennel.size(); i++) {
   kennel.get(i).bark();
}
```

It would be better if we could ensure that the ArrayList **only** contained Dogs in the first place

This is easily done because ArrayList uses a mechanism called **generics**.

We can specify the type allowed when we create the ArrayList.

Now Java will only allow us to add things of type **Dog**. So this line will force a **compile time** error

A Note About Primitives

```
ArrayList<Integer> numStore;
numStore = new ArrayList<Integer>();

numStore.add(new Integer(3));
numStore.add(new Integer(5));
numStore.add(new Integer(2));

int total = 0;

for(int i = 0; i < numStore.size(); i++) {
   total = total + numStore.get(i).intValue();
}

System.out.println("Total is " + total);
```

ArrayLists (and other collections in the API) can only store objects.

This means that when you want to store primitives you need to use wrapper objects.

This is a pain :-(

A Note About Primitives

```
ArrayList<Integer> numStore;
numStore = new ArrayList<Integer>();

numStore.add(3);
numStore.add(5);
numStore.add(2);

int total = 0;

for(int i = 0; i < numStore.size(); i++) {
   total = total + numStore.get(i);
}

System.out.println("Total is " + total);
```

ArrayLists (and other collections in the API) can only store objects.

This means that when you want to store primitives you need to use wrapper objects.

This is a pain :-(

Java 5 introduced **autoboxing**, a process where primitives are automatically cast to a wrapper where necessary.

And **unboxing**, where they can be cast back again too

Iterators

Design Patterns

- In Programming a neat and elegant way of solving a problem is sometimes called a **design pattern**
- The Java API uses a number of well-known design patterns
- Including the use of **iterators** to help you iterate over a collection

Back at the Kennel…

```
ArrayList<Dog> kennel = new ArrayList<Dog>();

kennel.add(new Dog("Rover"));
kennel.add(new Dog("Fido"));
kennel.add(new Dog("Patch"));
```

In our kennel example we used a for loop to iterate over the array

```
for(int i = 0; i < kennel.size(); i++) {
   kennel.get(i).bark();
}
```

Back at the Kennel…

```
ArrayList<Dog> kennel = new ArrayList<Dog>();

kennel.add(new Dog("Rover"));
kennel.add(new Dog("Fido"));
kennel.add(new Dog("Patch"));
```

```
for(int i = 0; i < kennel.size(); i++) {
   kennel.get(i).bark();
}
```

```
Iterator<Dog> it = kennel.iterator();

while(it.hasNext()) {
   it.next().bark();
}
```

In our kennel example we used a for loop to iterate over the array

We could instead use an iterator object.

Iterators are generic classes (like the ArrayList) and track our progress through a collection.

We can use **hasNext** to see if there are more elements

And **next** to get the next element (the iterator will automatically move to the next element).

Why are Iterators a useful pattern?

```
ArrayList<Dog> kennel = new ArrayList<Dog>();

kennel.add(new Dog("Rover"));
kennel.add(new Dog("Fido"));
kennel.add(new Dog("Patch"));


for(int i = 0; i < kennel.size(); i++) {
   kennel.get(i).bark();
}

Iterator<Dog> it = kennel.iterator();

while(it.hasNext()) {
   it.next().bark();
}
```

Why are Iterators a useful pattern?

```
ArrayList<Dog> kennel = new ArrayList<Dog>();

kennel.add(new Dog("Rover"));
kennel.add(new Dog("Fido"));
kennel.add(new Dog("Patch"));
```

```
for(int i = 0; i < kennel.size(); i++) {
   kennel.get(i).bark();
}

Iterator<Dog> it = kennel.iterator();

while(it.hasNext()) {
   it.next().bark();
}
```

1) They are neater, and neat code is easier to read and understand

2) They decouple the loop from the collection (notice that in the loop we do not reference the Arraylist at all)

This means we could pass the iterator to a method – and that method does not even need to know what the collection is!

Why are Iterators a useful pattern?

```
public void makeThemBark(Iterator<Dog> it) {

   while(it.hasNext()) {
        it.next().bark();
   }

}
```

1) They are neater, and neat code is easier to read and understand

2) They decouple the loop from the collection (notice that in the loop we do not reference the Arraylist at all)

This means we could pass the iterator to a method – and that method does not even need to know what the collection is!

Summary

- Arrays vs. ArrayLists
  - Declaration
  - Insertion
  - Access
  - Removal
- A Brief Introduction to Generics
  - Autoboxing and unboxing
- Iterator objects